

SIMATS ENGINEERING
DESIGN OF AN AI-BASED INTELLIGENT CAMPUS HELPDESK
AND FACILITY FAULT-DIAGNOSIS ASSISTANT

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

CSA17 - Artificial Intelligence

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

Kanish Kumar S (192511415)

Under the Supervision of

Dr.Nagarajan Babu

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

A.ASSIGNMENT INFORMATION

Attribute	Details
Department	Computer Science and Engineering
Programme	B.E / B.Tech
Course Code & Name	CSA17 - Artificial Intelligence
Maximum Marks	100
Course Outcomes	CO3, CO4, CO5

BLOOM'S TAXONOMY LEVELS:

- BL3 - Apply: Students will apply AI concepts to solve real-world problems
- BL4 - Analyse: Students will analyze system components and trade-offs
- BL5 - Evaluate: Students will evaluate solution effectiveness and alternatives
- BL6 - Create: Students will create an integrated intelligent system

B. ASSIGNMENT PROBLEM / CHALLENGE

Realistic Engineering/Computing Challenge:

Design of an AI-Based Intelligent Campus Helpdesk and Facility Fault-Diagnosis Assistant

Challenge Context: A university campus manages hundreds of daily facility complaints ranging from HVAC malfunctions to network outages. Manual triage and diagnosis is slow, error-prone, and creates a poor user experience as problem resolution is delayed.

Current Pain Points:

- Slow manual triage and routing of tickets
- Inconsistent diagnosis leading to repeat calls
- Difficulty in prioritizing critical vs. routine issues
- No learning from historical data to improve processes
- Inadequate escalation mechanisms for complex faults

Proposed AI Solution Capabilities:

- Accept and validate fault reports with symptom descriptions
- Apply knowledge-based reasoning (logic inference) for fault diagnosis
- Classify tickets using machine learning (decision trees, neural networks)
- Prioritize tickets based on impact and urgency
- Route tickets intelligently using reinforcement learning agents
- Provide transparent, explainable reasoning to users and staff

C. PROBLEM STATEMENT

Objective: Design and develop an AI-based Intelligent Campus Helpdesk and Fault-Diagnosis Assistant that integrates multiple AI paradigms to manage facility fault reports efficiently, accurately, and at scale.

System Functional Requirements:

Functional Requirement	Description
Fault Report Submission	Capture user ID, location, symptoms, priority. Validate and prevent duplicates.
Knowledge-Based Diagnosis	Use FOL/propositional logic to reason about faults. Apply resolution principle with forward/backward chaining.
ML-Based Classification	Train decision tree on historical tickets for category and priority classification.
Intelligent Routing	Use reinforcement learning to optimize ticket-to-staff assignment based on resolution time.
Explainability	Provide users and staff with reasoning traces and diagnosis confidence levels.

D. REQUIREMENTS AND CONSTRAINTS

D1. Functional Requirements (Detailed):

1. Fault Report Submission

- Capture and validate user and fault information
- Implement role-based access control (student/staff/admin)
- Prevent duplicate submissions within time window
- Assign ticket ID and track timestamp and status

2. Knowledge-Based Fault Diagnosis

- Encode facility domain knowledge as FOL/propositional logic rules
- Convert rules to CNF for resolution-based inference
- Apply resolution or chaining (forward/backward) to diagnose faults
- Request additional symptom information when facts are insufficient
- Return diagnosis with confidence score and explanation

3. Learning-Based Ticket Classification

- Train decision tree classifier on historical ticket data
- Use information gain or Gini impurity for splitting criteria
- Implement pruning and depth limits to prevent overfitting
- Classify tickets into issue categories and priority levels
- Continuously retrain as new data becomes available

4. Intelligent Routing Agent

- Learn optimal staff assignments using reinforcement learning (Q-learning/SARSA)
- Optimize for resolution time and first-contact resolution rate
- Handle multi-agent coordination for complex tickets
- Support manual override with learning feedback

D2. Performance Constraints:

- Diagnosis response time: < 5 seconds for interactive use
- Scalability: Support minimum 100 tickets, 20 fault categories, 5 departments

- Knowledge base: At least 50 fault diagnosis rules
- ML Model: Achieve minimum 80% classification accuracy
- System availability: 95% uptime during operational hours

D3. Technical Constraints:

- Language/Framework: Python 3.8+ with established ML libraries
- Logic Engine: Prolog (SWI-Prolog) or Python-based resolution engine
- ML Libraries: scikit-learn (decision trees), TensorFlow/PyTorch (if neural networks used)
- RL Library: OpenAI Gymnasium, Stable-Baselines3, or custom Q-learning
- Frontend: Flask/Django web interface or command-line interface
- Database: SQLite or PostgreSQL for ticket and rule storage

E. KEY LEARNING OUTCOMES & AI CONCEPT INTEGRATION:

AI Module/Concept	Learning Outcome	Application
Logical Agents & FOL	Understand knowledge representation and automated reasoning	Fault diagnosis via knowledge base encoding and resolution-based inference
Decision Trees & ML	Learn inductive learning, splitting criteria, and overfitting prevention	Automatic ticket classification by category and priority
Neural Networks/SVM	Understand deep learning and kernel methods	Escalation-risk prediction; fault pattern recognition
Reinforcement Learning	Master reward-based learning and policy optimization	Adaptive ticket-to-staff routing; optimization of resolution time
Agent Architectures	Design goal-oriented, reactive/deliberative agents	Multi-agent helpdesk coordinator; goal-based ticket resolution

F. ASSESSMENT RUBRIC & EVALUATION CRITERIA:

Assessment Criterion	Marks	Detailed Description
Problem Understanding	10	Clear problem identification, requirements definition, and scope articulation
Application of Course Knowledge	20	Meaningful integration of logical reasoning, learning, and agent concepts
Solution Design & Implementation	20	Well-structured modular architecture, clean code, working prototype
Use of Modern Tools & Techniques	10	Effective use of ML libraries, logic engines, web frameworks, documentation
Results, Testing & Validation	15	Comprehensive test suite, requirement validation, performance benchmarking
Analysis, Trade-offs & Justification	15	Thoughtful analysis of alternatives, explicit trade-off discussion
Ethics & Professional Responsibility	5	Discussion of ethics, data privacy, accessibility, and responsible AI
TOTAL	100	Full marks for comprehensive, well-integrated solution

G. PROPOSED SYSTEM ARCHITECTURE:

The Campus Helpdesk AI system integrates five key AI components into a unified architecture:

User Interface Layer: Web/CLI interface for ticket submission, status tracking, and result viewing

Ticket Management Module: Database and workflow engine for ticket lifecycle management

Logic-Based Diagnosis Engine: FOL knowledge base with resolution-based fault diagnosis

ML Classification Pipeline: Decision tree classifier for category and priority assignment

RL-Based Routing Agent: Q-learning agent for optimal staff assignment and continuous learning

H. DELIVERABLES:

1. AI System Prototype Source Code (GitHub repository with README and setup instructions)
2. Complete Assignment Report (all sections: problem formulation, design, implementation, results)
3. Execution Screenshots and Output Evidence (system interface, diagnosis examples, routing decisions)
4. Test Case Table with Results (minimum 20 test cases covering functional and edge cases)
5. Architecture and Design Diagrams (system architecture, data flow, decision tree visualization, RL policy)
6. Performance Analysis Report (benchmark results, classification accuracy, response times)
7. User Manual and Deployment Guide (installation, configuration, usage instructions)
8. Demonstration/Viva Presentation (system walkthrough, design decisions, lessons learned)

I. KEY IMPLEMENTATION CONSIDERATIONS

Knowledge Base Development:

The logic-based diagnosis engine requires careful domain modeling. Fault rules should be expressed in FOL as implications (antecedent $\rightarrow$ consequent). Example:

- $\forall \text{symptom, equipment: } (\text{symptom}=\text{no_power} \wedge \text{equipment}=\text{projector}) \rightarrow \text{fault}=\text{power_issue}$

The rule base should cover common faults (80/20 principle) and support abductive reasoning to handle cases where multiple faults could explain symptoms.

Feature Engineering for Classification:

Key features for ML model:

- Location type (classroom/lab/office/common area)
- Equipment category (HVAC/IT/electrical/structural)
- Symptom keywords and patterns
- Historical frequency of similar issues
- Time-of-day and day-of-week patterns

Reinforcement Learning State & Action Space:

RL Agent should optimize for:

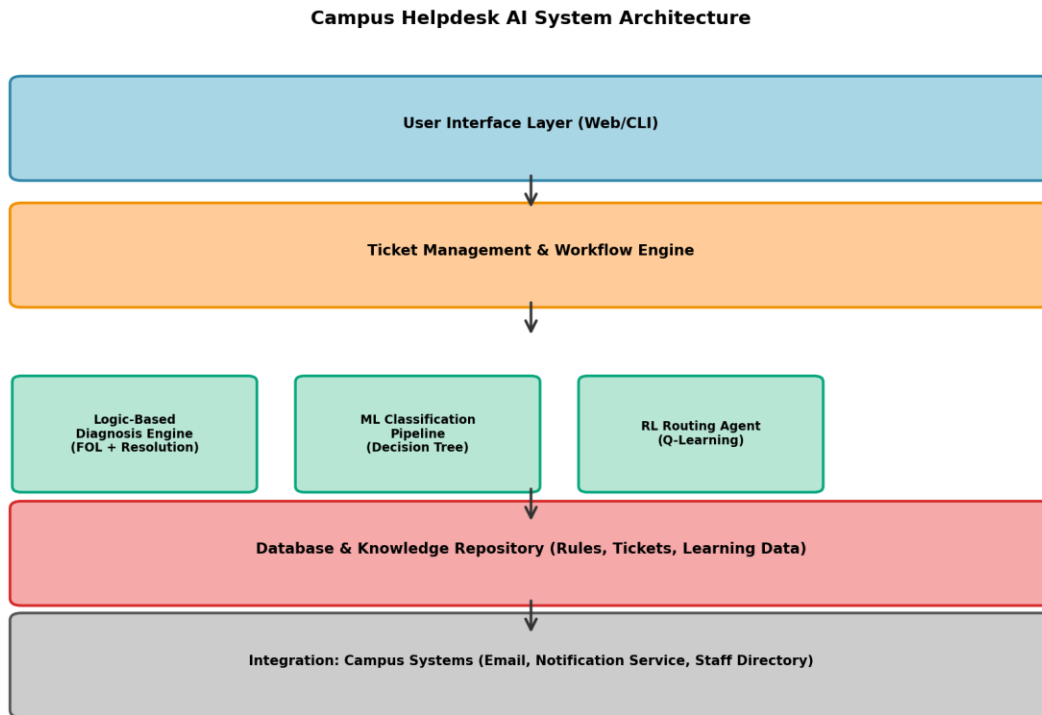
- State: (ticket_type, priority, complexity, staff_availability)
- Action: Assignment to staff member or team
- Reward: $-\text{resolution_time} + \text{bonus_for_first_contact_resolution}$

J. REFERENCES:

- Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Prentice Hall.
- Winston, P. H. (1992). Artificial Intelligence (3rd ed.). Addison-Wesley.
- Mitchell, T. M. (1997). Machine Learning. McGraw-Hill.
- Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.
- Scikit-learn: Machine Learning in Python. <https://scikit-learn.org/>
- TensorFlow: Open Source Machine Learning Framework. <https://www.tensorflow.org/>
- OpenAI Gymnasium: RL Environments. <https://gymnasium.farama.org/>
- SWI-Prolog: Logic Programming Language. <https://www.swi-prolog.org/>

G1. System Architecture Diagram

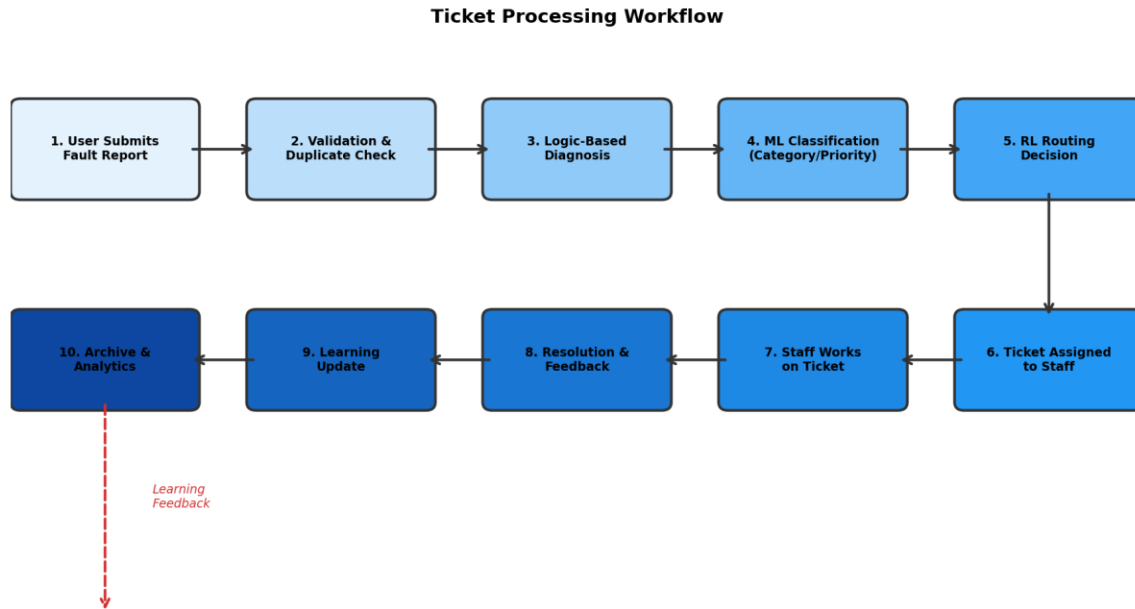
The following diagram illustrates the layered architecture of the Campus Helpdesk AI system:



The architecture comprises five integrated layers: (1) User Interface for ticket submission and status tracking, (2) Ticket Management module for workflow orchestration, (3) Three AI engines operating in parallel—Logic-based diagnosis, ML classification, and RL routing, (4) Unified database for knowledge and learning data, and (5) External system integration for notifications and staff coordination.

G2. Ticket Processing Workflow

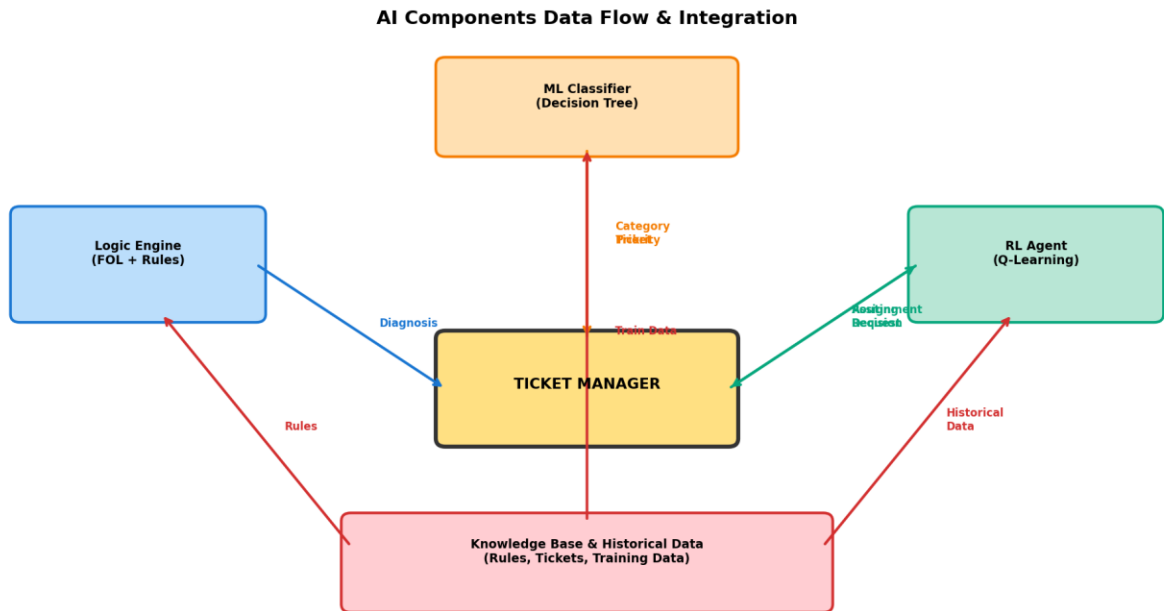
The following workflow diagram shows the complete lifecycle of a ticket from submission to resolution and feedback:



The workflow progresses through 10 stages: (1-4) Intake and analysis phases where validation, diagnosis, and classification occur, (5-8) Execution phases where routing, staff assignment, and resolution happen, (9-10) Learning phases where feedback is captured and data is archived. The dashed line shows the learning feedback loop that continuously improves the system through historical data.

G3. AI Components Data Flow

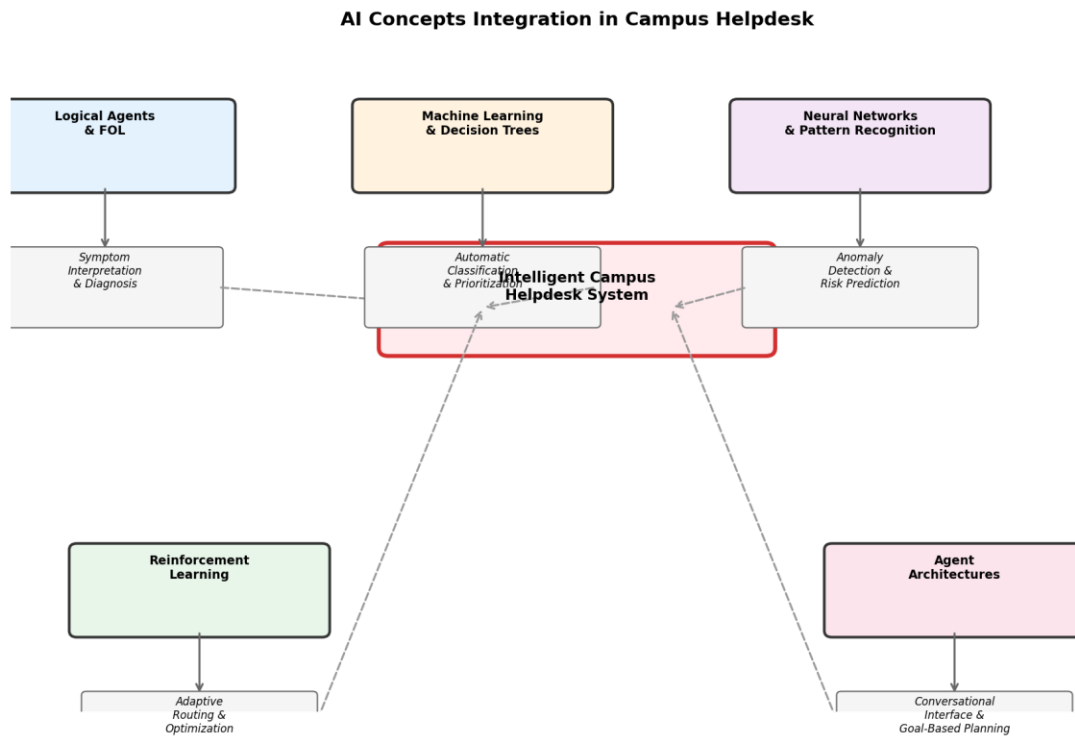
The following diagram illustrates how the three main AI components interact through the central Ticket Manager:



Data flows bidirectionally between components: The Logic Engine provides diagnoses based on rules, the ML Classifier processes ticket metadata for category and priority, and the RL Agent makes routing decisions. All components feed from and write to the Knowledge Base and Historical Data repository, enabling continuous learning.

G4. AI Concepts Integration Map

The following diagram maps AI course concepts to specific functions within the integrated system:



This diagram demonstrates how isolated AI concepts (left side) are operationalized as concrete functions (middle) and integrated into the unified Intelligent Campus Helpdesk System (center). The dashed lines show convergence, emphasizing that no single AI paradigm solves the problem alone—the solution requires thoughtful integration of multiple approaches.